

A Dual-Channel Contrastive Learning Framework for Anomaly Detection in Dynamic Graph Structures

Runshuo Liu
rslu@sdust.edu.cn
College of Computer Science and
Engineering, Shandong University of
Science and Technology
Qingdao, China

Chao Li*
lichao@sdust.edu.cn
College of Computer Science and
Engineering, Shandong University of
Science and Technology
Qingdao, China

Zhongying Zhao
zzysuin@163.com
College of Computer Science and
Engineering, Shandong University of
Science and Technology
Qingdao, China

Hui Zhou
zhouhui1026@foxmail.com
College of Computer Science and
Engineering, Shandong University of
Science and Technology
Qingdao, China

Qingtian Zeng
qtzeng@163.com
College of Computer Science and
Engineering, Shandong University of
Science and Technology
Qingdao, China

Abstract

Detecting data that deviates from normal behavior in dynamic graphs is an important hotspot. In the real world, anomaly detection is widely applied across various domains. However, existing anomaly detection methods typically assume static input graphs, failing to capture dynamic information in real-world scenarios. In addition, during the evolution of node representations over time, standard updating mechanisms lack the ability to selectively preserve important features. To address these issues, this paper constructs a dynamic graph anomaly detection framework (DyConAD) that enhances model perception of dynamic information through memory enhancement strategies and multi-scale similarity-based anomaly scoring mechanisms. Additionally, we also design a dynamic graph construction method that extracts time-aware shapelet and establishes spatiotemporal relationships between these shapelet. Experimental validation is conducted on four real-world datasets and three public datasets from the UCR Time Series Archive. Results demonstrate that our proposed method outperforms existing approaches across multiple datasets, achieving effective improvements in both detection accuracy and efficiency.

CCS Concepts

• **Theory of computation** → **Dynamic graph algorithms**; • **Computing methodologies** → *Feature selection*; Semi-supervised learning settings.

Keywords

Dynamic Graphs, Anomaly Detection, Time-aware Shapelet

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

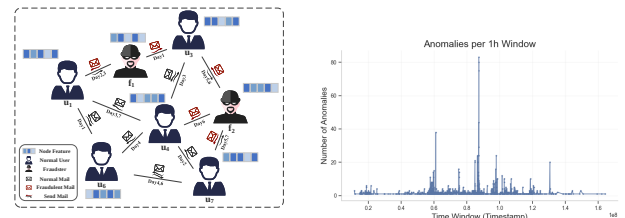
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792532>

ACM Reference Format:

Runshuo Liu, Chao Li, Zhongying Zhao, Hui Zhou, and Qingtian Zeng. 2026. A Dual-Channel Contrastive Learning Framework for Anomaly Detection in Dynamic Graph Structures. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792532>

1 Introduction

Anomaly detection in graph-structured data [3, 7] aims to identify behaviors that significantly deviate from normal patterns, offering valuable applications across multiple domains including network security, financial fraud detection, and social network analysis. However, existing anomaly detection research predominantly focuses on static graphs [9, 13], failing to adequately capture the dynamic characteristics of time series data. As illustrated in Figure 1, graph structures in real-world scenarios exhibit significant temporal dynamics [8]: Figure 1a demonstrates how fraudulent activities between accounts occur at different time points, forming temporally associated interaction patterns; Figure 1b reveals, through statistical analysis of the Bitcoinotc blockchain transaction network dataset, notable differences in the temporal distribution of attack frequencies by fraudsters targeting normal users.



(a) Fraudulent activities between accounts in dynamic graphs (b) Hourly frequency of anomalies in the bitcoinotc dataset

Figure 1: Temporal dynamics of graph structures in the real world.

In earlier works, researchers have explored how to integrate dynamic information in graphs for anomaly detection. For instance,

TADDY [12] employs positional encoding methods to assign specific temporal positions to each node, helping the model understand the relative position of nodes in time series. SemiADC adopts a self-learning approach, improving anomaly detection tasks through time series feature similarity and structural temporal correlation. However, these approaches merely incorporate time as a feature into the model, leading to insufficient temporal information modeling. To address these issues, SimpleDyG [24] reconceptualizes dynamic graphs as a sequence modeling challenge and introduces temporal alignment techniques to capture evolutionary patterns in dynamic graphs. SLADE [11] employs a memory generation strategy to create dynamic node representations from recent interactions, designing memory generation scores to identify the degree of deviation between short-term and long-term interactions. These methods ingeniously integrate temporal information from dynamic graphs into node embeddings through strategies such as temporal alignment and temporal storage. Despite the significant performance these methods have achieved in anomaly detection tasks on dynamic graphs, they still face the following challenges:

(1) Challenge 1: Insufficient Dynamic Feature Selection Capability. Most existing dynamic graph anomaly detection methods, such as SLADE[11] and GeneralDyG [26], employ standard recurrent neural network architectures like GRU for node representation updates. However, this conventional updating mechanism demonstrates critical limitations during the temporal evolution of node representations: it lacks both the capacity for selective retention of discriminative features and mechanisms for multi-level representation transformation and enhancement. Consequently, this leads to progressive attenuation of anomaly-related signals during propagation, substantially compromising detection performance.

(2) Challenge 2: Difficulty in Converting Time Series Data to Dynamic Graph Structures. Existing dynamic graph anomaly detection models cannot effectively utilize time series data. For instance, SAD [20] relies solely on traditional dynamic graph datasets for anomaly detection, while GeneralDyG [26] is limited to completing anomaly detection tasks on two blockchain-type datasets. However, the real world contains abundant time series data with rich anomalous patterns, and current research lacks effective methods to convert these time series data into dynamic graph structures for utilization.

To tackle the above challenges, we propose a dual-channel contrastive learning framework for anomaly detection in dynamic graph structures (DyConAD). The main contributions of this work include:

- We propose a memory enhancement strategy that enables multi-level transformation and enhancement of node memories.
- We introduce a time series converter capable of extracting time-aware temporal subsequences to construct dynamic graphs.
- We propose a multi-scale anomaly detector that employs self-supervised learning to identify deviations at varying levels of feature abstraction, significantly improving detection accuracy for complex anomalous patterns.
- We validate DyConAD on four real-world datasets and three public datasets from the UCR time series archive, demonstrating that our approach outperforms other state-of-the-art methods in terms of both performance and model efficiency.

2 Related Work

This section reviews related research work from two aspects: dynamic graph neural networks and dynamic graph anomaly detection.

2.1 Dynamic graph neural networks

Dynamic graph neural networks can be categorized into discrete-time dynamic graphs and continuous-time dynamic graphs based on their approach to temporal modeling. In discrete-time dynamic graph neural networks, the dynamic graph is first discretized into a sequence of time slices, with each static snapshot modeled independently, enabling effective learning of node embeddings. For example, DySAT [19] employs a multi-head attention mechanism to aggregate features across time slices, capturing long-term dependencies. EvolveGCN [15] proposes an innovative parameter evolution method that dynamically adjusts graph convolutional network parameters using recurrent neural networks rather than directly learning node embeddings. CTDNE [14] introduces temporal constraints to ensure that edge timestamps in walking paths strictly increase. Other representative works in this field include ROLAND [27] and others.

In continuous-time dynamic graph neural networks, the temporal evolution of graph structures is modeled as a continuous process, breaking through the limitations of traditional discrete time slices by directly processing event stream data through techniques such as differential equations and temporal encoding. GODE [16] combines graph neural networks with neural ODEs to capture the continuous temporal evolution of node features by solving ODEs. TGAT [4] employs temporal encoding and graph attention mechanisms to process continuous-time interaction events. TGN [18] captures the evolution of node states over time through memory modules and message passing mechanisms, suitable for continuous-time event sequences. CAWN [23] considers all possible temporal paths and weights them using temporal decay functions. Other representative works in this field include APAN [22], DyGFormer [28] and others.

2.2 Dynamic graph anomaly detection

Dynamic graph anomaly detection aims to discover abnormal patterns in graph structures or node behaviors that change dynamically over time. Traditional methods are primarily designed for static graphs. For example, DOMINANT uses a graph autoencoder module to reconstruct graph topology and attributes, identifying anomalies based on reconstruction errors. GDN [5] first explores the new problem of few-sample network anomaly detection, achieving more effective anomaly detection by integrating information from multiple networks. Other representative works include Radar, SemiGNN [29], ConOut [25], and others. As research deepens, researchers have found that focusing only on static graph structures while ignoring temporal evolution characteristics leads to information loss, thereby affecting the accuracy and timeliness of anomaly detection.

To incorporate dynamic information from the real world, researchers have shifted their attention to dynamic graph anomaly detection. SAD [20] effectively extracts node features using a self-attention mechanism and applies them to graph anomaly detection

and representation learning tasks. SimpleDyG [24] employs a simplified architecture, reducing dependency on complex graph structures while focusing on learning node features to improve anomaly detection speed. SLADE [11] effectively identifies anomalous events in dynamic graphs through self-supervised mechanisms, reducing reliance on labeled data. GeneralDyG [26] achieves good results in dynamic graph anomaly detection tasks by sampling graphs and sequentially extracting structural and temporal features. Other representative works in this field include StrGNN [1], SEDANSPOT [6] and others.

3 Preliminaries

In this section, we formally define some important concepts and problems related to dynamic graph neural networks.

Continuous-time Dynamic Graph. A continuous-time dynamic graph can record the complete temporal information of a dynamic network, represented as $G = \{(v_i, v_j, t, \Delta t, e_{ij}(t)); i, j = 1, 2, \dots, n\}$, where v_i, v_j denote nodes in the network, $t, \Delta t$ represent the appearance time and duration of the connection, respectively, and $e_{ij}(t)$ represents the edge.

Dynamic Graph Neural Networks. Given a dynamic graph G , an original node u , a target node v , a timestamp t , and historical interaction information $\{(u', v', t') \mid t' < t\}$, representation learning on dynamic graphs aims to design a mapping function f to learn time-aware representations $h_u^t \in \mathbb{R}^d$ and $h_v^t \in \mathbb{R}^d$ for u and v , where d is the dimension.

Dynamic Graph Anomaly Detection. The goal of dynamic graph anomaly detection is to construct a function $f : G(t) \rightarrow \mathbb{R}^{|V(t)|}$, such that for any node $v \in V(t)$, an anomaly score function $s_v(t)$ is designed to identify node, edge, or global anomalies.

4 Methodology

The core challenge of anomaly detection in dynamic graphs lies in: how to capture interaction patterns of nodes in dynamically changing time series and identify anomalous nodes that deviate from normal behavior. To address this, we propose a self-supervised dynamic graph anomaly detection model based on memory enhancement, called DyConAD. This method enhances memory through node dynamic features, modeling long-term interaction patterns and short-term behavioral changes of nodes, and combines self-supervised learning to achieve efficient anomaly detection without labels, as shown in the model framework in Figure 2.

4.1 Time Series Transformer

4.1.1 Time-aware shapelet extraction. Shapelets are discriminative subsequences within time series. To identify discriminative subsequences with temporal sensitivity, we define local temporal weights and global temporal weights to quantitatively measure the temporal effects of subsequences at different levels. Specifically:

(Local Temporal Weights) Through Dynamic Time Warping (DTW) operations, we assign weights to points along the alignment path, amplifying discriminative differences at critical time points. For each candidate shapelet $v \in \mathbb{R}^l$, we introduce a weight vector

$w = [w_1, \dots, w_l]$:

$$\hat{d}(v, s|w) = \sqrt{\sum_{k=1}^p w_{a(k)} \cdot (v_{a(k)} - s_{a^*(k)})^2}, \quad (1)$$

where $a^{(k)}$ represents the index of the optimal alignment path obtained from DTW calculation. This path allows the shapelet to elastically align with the target subsequence along the time axis, while the weight $w_{a(k)}$ further emphasizes positions with high discriminative power in the alignment points.

(Global Temporal Weights) To capture periodic or phase changes in temporal patterns, we divide the time series t_i into m consecutive segments $\{s_1, \dots, s_m\}$, and assign a global weight u_k to each segment. The global distance formula is:

$$\hat{D}(v, t|w, u) = \min_{1 \leq k \leq m} u_k \cdot \hat{d}(v, s_k|w), \quad (2)$$

The loss function L is designed to maximize the distinction between positive and negative sample pairs:

$$L = -E[g(S_{pos}, S_{neg})] + \lambda \|w\|_2^2 + \epsilon \|u\|_2^2, \quad (3)$$

where the first term $-E[g(S_{pos}, S_{neg})]$ measures the distributional difference between positive sample pairs (shapelets from the same class of time series) and negative sample pairs (shapelets from different classes) based on KL divergence, forcing the model to learn more discriminative weight allocations. The regularization terms $\lambda \|w\|_2^2 + \epsilon \|u\|_2^2$ prevent weights from overly concentrating on a few time points or time periods, enhancing the model's generalization ability.

4.1.2 Evolution dynamics of continuous shapelet. To model the temporal characteristics of pattern transitions in time series, we propose a dynamic evolution graph. This method captures both short-term fluctuations and long-term trends in pattern evolution through time window segmentation and dynamic weight updating strategies.

The time series data stream is divided into overlapping windows $\{W_1, \dots, W_K\}$, with window length τ and step size $\Delta\tau$ adaptively selected based on data frequency. For each segment s in window W_K , we filter the associated Shapelet set:

$$C(s) = v_j | \hat{D}(v_j, s) \leq \mu_k - 1.5\sigma_k, \quad (4)$$

where μ_k and σ_k are the mean and standard deviation of distances within the window.

We define a time decay factor $\gamma = e^{-\beta\Delta t}$, and the transition weight between adjacent segments s_t and s_{t+1} as:

$$w_{p,q}^{(k)} = \gamma \cdot \frac{\exp(-\hat{D}(v_p, s_t) - \hat{D}(v_q, s_{t+1}))}{\sum \exp(-\hat{D}(v', s_t))}, \quad (5)$$

The implicit physical meaning is: if Shapelet v_p matches the current segment s_t well, and v_q matches the next segment s_{t+1} , then the transition probability $w_{p,q}$ is higher.

For global graph fusion, we aggregate subgraphs from each window through temporal smoothing using a Gaussian kernel function:

$$\bar{w}_{p,q} = \sum_{k=1}^K w_{p,q}^{(k)} \cdot \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(t-t_k)^2}{2\sigma^2}}, \quad (6)$$

We define the dynamic graph $G(t) = (V(t), E(t), W(t))$, with the update rule:

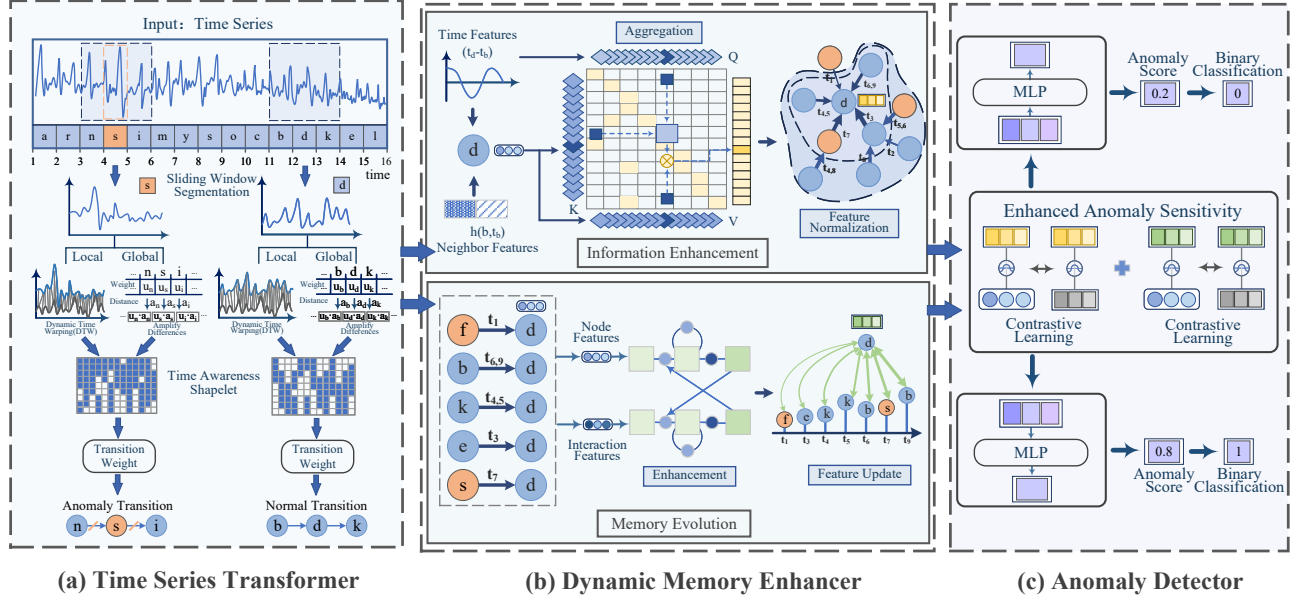


Figure 2: The overall framework of DyConAD. First, the (a) Time Series Transformer constructs a dynamic graph from time series data, then the (b) Dynamic Memory Enhancer strengthens the perceived dynamic features, and finally the (c) Anomaly Detector identifies interaction anomalies.

$$W(t)_{p,q} = \begin{cases} (1 - \eta)W(t - \Delta t)_{p,q} + \eta \tilde{w}_{p,q}^{(k)} & \text{for existing edges} \\ \tilde{w}_{p,q}^{(k)} & \text{for new edges} \end{cases} \quad (7)$$

where $\eta \in [0, 1]$ is the smoothing coefficient controlling the proportion of historical weight retention, thus constructing the dynamic graph.

4.2 Dynamic Memory Enhancer

4.2.1 Information Enhancement. In dynamic graphs, node structure information patterns evolve over time. This work proposes an adaptive attention module that fuses temporal encoding with dynamic attention weights to continuously update node structure memory.

Let v be the current center node, t_v be the timestamp of the center node, and $N_v(t_v)$ be the set of neighbors connected to node v before time t_v . $e_{(v,u)}$ represents the edge feature between center node v and target node u , $f(v, t_v)$ is the feature representation of node v at time t_v , and $h^{(l)}(v, t_v)$ is the temporal feature representation of node v in the l -th convolution layer.

To capture structural information in the overall evolution process of dynamic graphs, we define the initial feature of a node as its feature representation in the 0-th layer:

$$h^{(0)}(v, t_v) = f(v, t_v), \quad (8)$$

To compute the temporal difference feature $\phi(t_v - t_u)$ between center node v and target node u , we utilize Bochner's theorem through a time encoder, using cosine functions to transform mapping values into harmonic features that are time-sensitive and periodically

varying:

$$\phi(t_v - t_u) = \cos((t_v - t_u) \cdot w_k + p_k), \quad (9)$$

where w_k represents a fixed base frequency sequence, defined as $w_k = 1/10^i$, $i = 0, 1, \dots, n$, with n being the time dimension. It represents a frequency range from low to high frequencies, serving as the base frequency for temporal encoding. p_k represents a phase offset vector used to adjust the initial offset in different dimensions, with an initial value of 0.

To obtain the feature representation of node v in the l -th layer ($l \geq 1$), we need to aggregate the representations and temporal information of its neighbors. Here, we choose to use an attention mechanism to calculate the weighted contribution of neighbor features to the query node:

$$\begin{aligned} Q(t) &= h^{(l-1)}(v, t_v)W_Q, \\ K(t) &= [h^{(l-1)}(u, t_u), e_{(v,u)}, \phi(t_v - t_u)]W_K, \\ V(t) &= [h^{(l-1)}(u, t_u), e_{(v,u)}, \phi(t_v - t_u)]W_V, \end{aligned} \quad (10)$$

where $Q(t)$ represents the weighted feature of the center node, $K(t)$ and $V(t)$ represent the weighted features of neighbor node u , $h^{(l-1)}(v, t_v)$ represents the actual feature of the center node in the previous layer, $h^{(l-1)}(u, t_u)$ represents the actual feature of the neighbor node in the previous layer, W_Q, W_K, W_V are weight matrices, $e_{(v,u)}$ is the edge feature, and $\phi(t_v - t_u)$ is the temporal difference feature.

The formula for calculating the contribution weight of neighbor node u to center node v is as follows:

$$\alpha_u = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V, \quad (11)$$

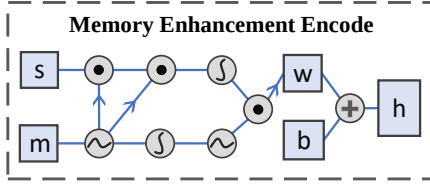


Figure 3: Memory enhancement unit.

where $\sqrt{d}$ is a scaling factor for data normalization. Finally, the feature embedding representation of node v in the l -th layer ($l \geq 1$) after capturing the structure is:

$$h_{\text{struct}}^l(v, t_v) = \sum_{u \in N_v(t_v)} \alpha_u \cdot (h^{(l-1)}(u, t_u) + e_{(v,u)}, \phi(t_v - t_u)), \quad (12)$$

4.2.2 Memory Evolution. The memory evolution module aims to capture the dynamic evolutionary characteristics of node behavior by continuously updating the short-term interaction patterns of nodes to reflect their latest states.

For a newly arrived edge event $e_k = (u, v, t_k)$, the time memory module generates two messages m_u and m_v for the source node u and target node v respectively. The message computation process is as follows:

$$\begin{aligned} m_u &= \text{MLP}([s'_u \parallel \phi(t_k - t_u^-)]), \\ m_v &= \text{MLP}([s'_v \parallel \phi(t_k - t_v^-)]), \end{aligned} \quad (13)$$

where s'_u and s'_v are the current memory vectors of the nodes, $\phi(\cdot)$ is the time encoding function, and t_u^- and t_v^- represent the timestamps of the last updates for nodes u and v . The MLP layer fuses neighbor information with temporal difference features into low-dimensional messages.

The dynamic features of nodes are updated through memory enhancement units:

$$\begin{aligned} h_{u_{\text{time}}} &= \text{EnMem}(m_u, s'_u), \\ h_{v_{\text{time}}} &= \text{EnMem}(m_v, s'_v), \end{aligned} \quad (14)$$

As shown in Figure 3, the memory enhancement unit integrates the most important interaction information between nodes and preserves long-term behavioral patterns through multi-level transformation and enhancement. The updated memory vectors $h_{u_{\text{time}}}$ and $h_{v_{\text{time}}}$ will serve as the new states of the nodes until the next interaction occurs.

4.3 Anomaly Detector

The anomaly detector module aims to dynamically assess the degree of abnormality in node behavior through a self-supervised contrastive learning strategy. This module effectively identifies mutations in node states and long-term pattern anomalies by combining the spatiotemporal deviation of structural memory and temporal memory.

4.3.1 Loss Calculation. Based on the outputs from the structural memory channel and temporal memory channel, a cross-view contrastive loss is defined to enhance anomaly sensitivity. For node v with embedding representations $h_v^{(\text{struct})}(t)$ and $h_v^{(\text{time})}(t)$ at time

t , positive and negative sample pairs are constructed through a dynamic memory bank M that stores historical normal pattern prototypes:

$$L_{\text{cont}} = -\log \frac{\exp(\text{sim}(h_v^{(\text{time})}, h_v^{(\text{struct})})/\tau)}{\sum_{m \in M} \exp(\text{sim}(h_v^{(\text{time})}, m)/\tau)}, \quad (15)$$

where $\text{sim}(\cdot)$ is the cosine similarity function, and τ is the temperature coefficient. The memory bank M is maintained through a momentum update strategy:

$$m_k \leftarrow \gamma m_k + (1 - \gamma) h_v^{(\text{norm})}, \forall k \in N(v), \quad (16)$$

This loss forces normal nodes to maintain representation consistency across the structure-time dual channels, while anomalous nodes exhibit significantly increased contrastive loss due to pattern deviation.

4.3.2 Anomaly Score Generation. Through the weighted fusion of temporal contrastive scores and memory reconstruction scores, fine-grained anomaly scores are generated. The scoring mechanism includes the following two core components:

(Time Contrastive Score) This score measures the degree of mutation in a node's long-term interaction patterns. For the state of node v_i at time t , the cosine distance between its current memory vector $m_i(t)$ and its previous version $m'_i(t)$ is calculated:

$$S_{tc}(v_i, t) = 1 - \text{cosine}(m_i(t), m'_i(t)), \quad (17)$$

where $\text{cosine}(m_i(t), m'_i(t))$ computes the cosine similarity. Memory vectors of normal nodes should evolve smoothly over time, while anomalous nodes will show significantly elevated scores due to behavioral pattern mutations.

(Memory Generation Score) This score evaluates the deviation degree between short-term interaction patterns and long-term patterns. It is calculated through the difference between the momentum-updated reconstructed vector $\hat{m}_i(t)$ and the current memory vector:

$$S_{mg}(v_i, t) = 1 - \text{cosine}(\hat{m}_i(t), m_i(t)), \quad (18)$$

Normal nodes should be able to accurately reconstruct memory through recent interactions, while anomalous nodes will exhibit significant differences between generated memory and actual memory.

The final anomaly score is the weighted harmonic mean of both:

$$S(v_i, t) = \frac{1}{4}(S_{tc}(v_i, t) + S_{mg}(v_i, t)), \quad (19)$$

This score is normalized to the $[0,1]$ interval.

5 Experiments

In this section, we first introduce the experimental settings, and then analyze the performance of DyConAD by answering the following Research Questions.

- **RQ1:** How does the proposed model perform on anomaly detection tasks compared to state-of-the-art baseline methods?
- **RQ2:** Is there a significant difference between the normal score distribution and anomaly score distribution of the model?
- **RQ3:** How does the proposed model compare to state-of-the-art baseline methods in terms of time and space complexity?

- **RQ4:** Can the model provide an interpretable transformation process from temporal data to dynamic graphs?

5.1 Experimental Settings

Datasets. This paper employs seven datasets to evaluate the model’s effectiveness. These include two classic dynamic graph datasets (Wikipedia, Reddit), two financial networks of Bitcoin user trust relationships (Bitcoin-alpha, Bitcoin-OTC), and three public datasets from the UCR time series archive (Earthquakes, WormsTwoClass, Strawberry).

Baselines. We compare our proposed method with four categories of anomaly detection methods. Traditional anomaly detection methods: SedanSpot [6], MIDAS [17], F-FADE [2]. Incremental detection methods: Anoedge. Dynamic graph embedding methods: JODIE [10], DyRep [21], TGAT [4], TGN [18]. Dynamic memory storage methods: SAD [20], SLADE [11].

5.2 Anomaly Detection (RQ1)

How does the proposed model perform on anomaly detection tasks compared to state-of-the-art baseline methods? We used AUC as the evaluation metric to assess the model’s performance on anomaly detection tasks.

As shown in Table 1, DyConAD achieves the best or near-best detection performance across all datasets, fully demonstrating the effectiveness and robustness of the proposed method. On classical dynamic graph datasets, DyConAD exhibits significant advantages: on the Wikipedia dataset, it achieves an AUC of $88.20 \pm 0.26\%$, improving by 0.45 percentage points over the second-best method SLADE ($87.75 \pm 0.68\%$) and by 0.73 percentage points over the traditional dynamic graph embedding method TGN ($87.47 \pm 0.22\%$); on the Reddit dataset, it obtains an AUC of $71.38 \pm 0.43\%$, significantly outperforming TGN ($67.16 \pm 1.03\%$) by 4.22 percentage points. On blockchain financial networks, DyConAD’s performance is particularly outstanding: on the Bitcoin-alpha dataset, it achieves the highest AUC of $76.39 \pm 0.21\%$, improving by 0.52 percentage points over SLADE; on the Bitcoin-OTC dataset, it obtains the best performance of $76.87 \pm 0.28\%$, improving by 0.64 percentage points over TGN. These results indicate that DyConAD can more effectively identify abnormal transaction patterns and fraudulent behaviors in financial transaction networks, which has important practical application value for blockchain security monitoring.

Compared with different types of baseline methods, DyConAD demonstrates comprehensive performance advantages. Compared with traditional anomaly detection methods (SedanSpot, MIDAS, F-FADE, Anoedge-I), DyConAD shows significant improvements across all datasets, with improvement margins ranging from 10% to 40%, indicating that deep learning methods have obvious advantages in capturing complex spatiotemporal patterns. Compared with dynamic graph embedding methods (JODIE, Dyrep, TGAT, TGN), DyConAD maintains a performance advantage of 1%-6%, demonstrating that the contrastive learning framework can better learn discriminative representations of nodes. Even compared with the well-performing dynamic memory storage method SLADE, DyConAD still maintains a performance improvement of 0.5%-1% on most datasets, achieving the best performance on 5 out of 7 datasets. On pure time series datasets, DyConAD also maintains competitive

advantages: it achieves $74.62 \pm 0.14\%$ on the EQS dataset and obtains the best performance of $75.57 \pm 0.22\%$ on the WTC dataset. Although it is slightly lower than TGN and SLADE on the STB dataset, it still ranks third, demonstrating good temporal modeling capability.

Several key findings can be drawn from the experimental results: First, DyConAD performs excellently on three different types of datasets (classical dynamic graphs, blockchain networks, and time series), proving that the model has good cross-domain generalization capability; second, on structurally complex dynamic graph networks, DyConAD’s advantages are more pronounced, indicating that the model can effectively capture spatiotemporal anomaly patterns in complex networks; third, from the standard deviation data, it can be seen that DyConAD has relatively small standard deviations on most datasets (e.g., Wikipedia: ± 0.26 , Bitcoin-OTC: ± 0.28), indicating that the model has good stability and reproducibility. In summary, the experimental results fully validate the effectiveness of the DyConAD method. By combining dynamic graph neural networks and contrastive learning framework, DyConAD provides a solution for dynamic graph anomaly detection.

5.3 Anomaly Visualization (RQ2)

Do the normal and anomalous score distributions show significant differences? To address this question, we designed visualization experiments of anomaly detection result distributions, revealing the performance characteristics of the DyConAD model through differential distributions, with specific analyses as follows:

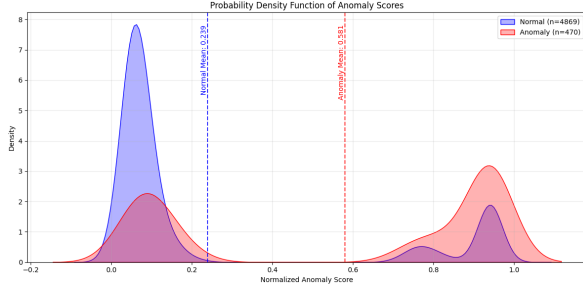
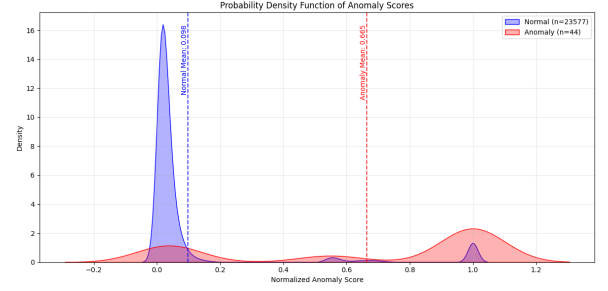
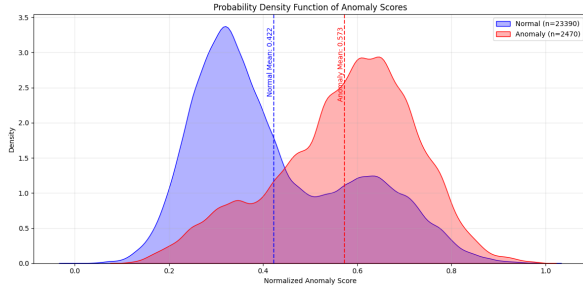
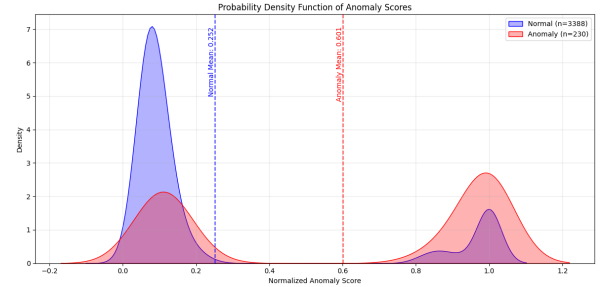
As shown in Figure 4, from the perspective of overall distribution characteristics, the anomaly scores of normal nodes and anomalous nodes exhibit significant separability. We conducted detailed distribution analysis on four representative datasets, each demonstrating unique and meaningful pattern characteristics. Normal nodes across all datasets present left-skewed distributions with peaks concentrated in low-score regions, while anomalous nodes generally exhibit right-skewed or bimodal distributions with the main mass concentrated in high-score regions.

In the Bitcoin-OTC dataset, which contains 4,869 normal nodes and 470 anomalous nodes, normal transaction nodes have a mean score of 0.239, presenting a highly concentrated left-skewed distribution with a peak density of approximately 7.8. The Bitcoin-alpha dataset (3,388 normal nodes and 230 anomalous nodes) demonstrates a similar but more polarized distribution pattern, with normal nodes having a mean of 0.252 and anomalous nodes having a mean of 0.601, with a mean difference of 0.349 (58.1%), showing even better separation than Bitcoin-OTC. Both blockchain datasets indicate that the model can effectively identify financial fraudulent transactions.

The Wikipedia dataset records Wikipedia editing behavior, containing 23,577 normal nodes and only 44 anomalous nodes, exhibiting extremely imbalanced yet highly separable distribution characteristics. Normal editing nodes have a mean score of 0.098 with a peak density of approximately 16.5, the highest density value among all datasets, with the main peak concentrated in the 0.05-0.15 extremely low-score range. Anomalous editing nodes have a mean score of 0.665, which is 6.78 times that of normal nodes, mainly concentrated in the 0.6-1.0 range with a peak density of approximately 2.3.

Table 1: Experimental Results for Anomaly Detection (AUC)

Method	Classic Dynamic		Blockchain		Time Series		
	Wikipedia	Reddit	Bitcoin-alpha	Bitcoin-OTC	EQS	WTC	STB
SedanSpot	82.88±1.54	58.97±1.85	69.09±0.76	71.71±0.73	68.20±2.10	66.45±1.80	63.15±3.50
MIDAS	62.92±3.53	59.94±0.95	64.57±0.11	62.16±1.99	61.30±1.20	60.88±0.75	58.74±2.30
F-FADE	44.88±0.02	49.79±0.05	53.57±0.06	51.12±0.03	50.25±0.08	52.33±0.12	48.67±0.15
Anoedge-l	47.38±0.32	48.47±0.46	62.52±0.24	65.99±0.1	63.15±0.35	64.80±0.28	60.42±0.50
JODIE	85.75±0.17	61.47±0.58	73.53±1.26	69.13±0.96	71.20±0.90	70.65±1.10	66.88±1.80
Dyrep	85.68±0.34	63.42±0.62	73.30±1.51	70.76±0.70	72.15±1.20	71.33±0.95	67.25±1.60
TGAT	83.24±1.11	65.12±1.65	71.67±0.90	68.33±1.23	69.80±0.85	68.95±1.05	65.40±2.10
TGN	87.47±0.22	67.16±1.03	69.90±0.99	<u>76.23±0.23</u>	73.50±0.60	74.15±0.45	70.20±0.80
SAD	86.15±0.63	68.45±1.27	68.56±3.17	64.67±4.97	66.30±2.50	67.85±1.90	63.75±3.80
SLADE	<u>87.75±0.68</u>	72.19±0.60	<u>75.87±0.28</u>	75.80±0.19	74.43±0.16	75.03±1.60	<u>69.45±0.36</u>
DyConAD	88.20±0.26	<u>71.38±0.43</u>	76.39±0.21	76.87±0.28	74.62±0.14	75.57±0.22	68.93±0.23

**(a) Bitcoin-otc****(b) Wikipedia****(c) WTC****(d) Bitcoin-alpha****Figure 4: Anomaly Visualization**

The WTC dataset records the movement behavior patterns of *C. elegans*, containing 23,390 normal nodes and 2,470 anomalous nodes, having the highest proportion of anomalous samples (9.6%) among the four datasets. Normal behavior nodes have a mean score of 0.422, presenting a broad normal distribution with a peak density of approximately 3.4, reflecting the natural variability of biological behavior. Anomalous behavior nodes have a mean score of 0.573, with a mean difference of 0.151 (35.8%), the smallest among all datasets.

Comprehensive analysis shows that DyConAD achieves effective differentiation of anomaly scores through spatiotemporal joint modeling.

5.4 Time and Space Complexity (RQ3)

How does the proposed model perform in terms of time and space complexity compared to state-of-the-art baseline methods? In the anomaly detection task, we calculated the memory consumption and training time per epoch on the Wikipedia dataset. To ensure

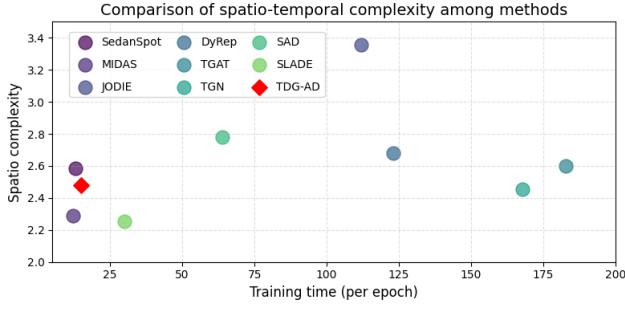


Figure 5: Time Complexity Experiment

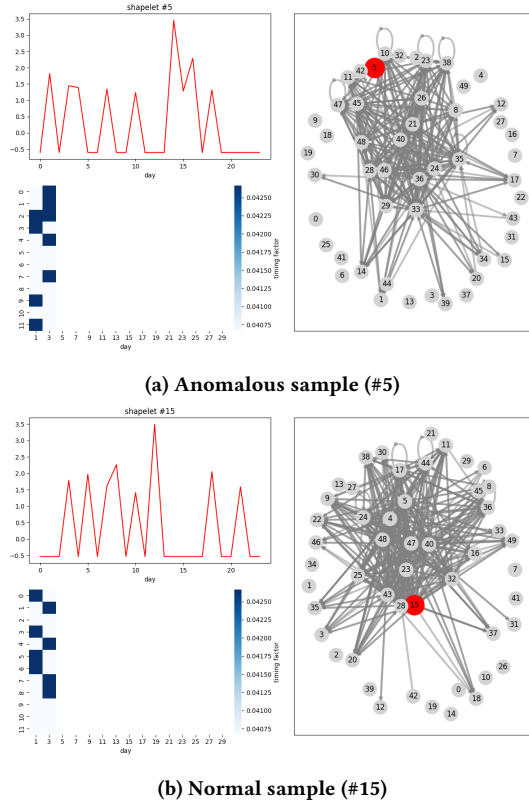


Figure 6: Visualization analysis of time series to dynamic graph conversion.

the fairness of the experiments, all experiments were conducted on a Windows system with an RTX-3060 GPU. The experimental results are shown in the figure.

In dynamic graph anomaly detection tasks, time and space complexity are important metrics for measuring model efficiency. As shown in Figure 5, DyConAD demonstrates excellent time and space complexity performance on the Wikipedia dataset, with a training time of approximately 15 seconds per epoch and memory consumption of around 2.4GB, significantly outperforming most

baseline models. In contrast, traditional methods such as SedanSpot and MIDAS have lower time complexity but cannot effectively capture complex dynamic patterns in dynamic graphs. Overall, DyConAD achieves a good balance between efficiency and performance in terms of time and space complexity, providing an efficient and practical solution for dynamic graph anomaly detection tasks.

5.5 Time Series to Dynamic Graph (RQ4)

Can the model provide interpretable anomaly detection results when converting time series data to dynamic graphs? To verify the interpretability of the model in the process of converting time series data to dynamic graphs, this section presents a visual analysis of the experimental results.

As shown in Figure 6, the upper left corner displays the original time series data for #5 and #15, intuitively reflecting the typical pattern characteristics within that time period; the lower left corner shows the temporal awareness capability of the time subsequence, revealing the distribution of importance weights of this pattern across different time periods; the right side demonstrates the dynamic evolutionary relationships between this pattern and other patterns by transforming the relationships between various time subsequences into a graph form. From an interpretability perspective, through the display of the original sequence and timing factors, we can intuitively understand the specific form of anomalous patterns and their influence changes in the time dimension. For normal records, there exists a clear path in the graph for the transition of their time subsequences. However, for anomalous data, no path exists.

6 Conclusion

In conclusion, this paper proposes a dual-channel contrastive learning framework for anomaly detection in dynamic graph structures (DyConAD) model. First, addressing the insufficient dynamic feature selection capability of existing dynamic graph anomaly detection methods, we designed a memory enhancement strategy that can perform multi-level transformation and enhancement of node memory. Second, to address the issue of limited data source types used in existing dynamic graph anomaly detection models, we constructed a time series converter capable of extracting time subsequences with temporal awareness to construct dynamic graphs. Finally, through a multi-scale anomaly detector, self-supervised contrastive learning, and other approaches, we achieved comprehensive capture of anomalous patterns. funding sources and other support, and thanks to individuals and groups that assisted in the research and the preparation of the work should be included in an acknowledgment section, which is placed just before the reference section in your document.

Acknowledgments

This research is supported by the National Natural Science Foundation of China (Grant No. 62472263, 52374221), the Natural Science Foundation of Shandong Province (Grant No. ZR2024MF034). Humanities and Social Science Research Project of the Ministry of Education, China (Grant No. 24YJAZH058), the CAHE 2024 Higher Education Scientific Research Planning Project (Grant No. 24XX0101).

References

- [1] Lei Cai, Zhengzhang Chen, Chen Luo, Jiaping Gui, Jingchao Ni, Ding Li, and Haifeng Chen. 2021. Structural Temporal Graph Neural Networks for Anomaly Detection in Dynamic Graphs. In *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management*. ACM, 3747–3756.
- [2] Yen-Yu Chang, Pan Li, Rok Soscic, M. H. Afifi, Marco Schweighauser, and Jure Leskovec. 2021. F-FADE: Frequency Factorization for Anomaly Detection in Edge Streams. In *WSDM '21, The Fourteenth ACM International Conference on Web Search and Data Mining*. ACM, 589–597.
- [3] Guodong Chen, Jesús Arroyo, Avanti Athreya, Joshua Cape, Joshua T. Vogelstein, Youngser Park, Christopher M. White, Jonathan Larson, Weiwei Yang, and Carey E. Priebe. 2025. Multiple network embedding for anomaly detection in time series of graphs. *Comput. Stat. Data Anal.* 203 (2025), 108070.
- [4] Chaofan Dai, Qideng Tang, and Huahua Ding. 2024. TGAT: Temporal Graph Attention Network for Blockchain Phishing Scams Detection. In *International Conference on Computer, Information and Telecommunication Systems*. IEEE, 1–7.
- [5] Ailin Deng and Bryan Hooi. 2021. Graph Neural Network-Based Anomaly Detection in Multivariate Time Series. In *Thirty-Fifth AAAI Conference on Artificial Intelligence*. AAAI Press, 4027–4035.
- [6] Dhivya Eswaran and Christos Faloutsos. 2018. SedanSpot: Detecting Anomalies in Edge Streams. In *IEEE International Conference on Data Mining*. IEEE Computer Society, 953–958.
- [7] Cong Gao, Hongye Ma, Qingqi Pei, and Yanping Chen. 2025. Dynamic graph-based graph attention network for anomaly detection in industrial multivariate time series data. *Appl. Intell.* 55, 6 (2025), 517.
- [8] Rong Gao, Zhiwei Chen, Xinyun Wu, Yonghong Yu, and Li Zhang. 2025. Dynamic deep graph convolution with enhanced transformer networks for time series anomaly detection in IoT. *Clust. Comput.* 28, 1 (2025), 15.
- [9] Jinrui Han, Zhen Chen, Di Zhou, Bing Hu, Tangbin Xia, and Ershun Pan. 2025. Unsupervised motion-based anomaly detection with graph attention networks for industrial robots labeling. *Eng. Appl. Artif. Intell.* 146 (2025), 110298.
- [10] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 1269–1278.
- [11] Jongha Lee, Sunwoo Kim, and Kijung Shin. 2024. SLADE: Detecting Dynamic Anomalies in Edge Streams without Labels via Self-Supervised Learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 1506–1517.
- [12] Yixin Liu, Shirui Pan, Yu Guang Wang, Fei Xiong, Liang Wang, Qingfeng Chen, and Vincent Cheng-Siong Lee. 2023. Anomaly Detection in Dynamic Graphs via Transformer. *IEEE Trans. Knowl. Data Eng.* 35, 12 (2023), 12081–12094.
- [13] Xiaoxiao Ma, Fanzhen Liu, Jia Wu, Jian Yang, Shan Xue, and Quan Z. Sheng. 2025. Rethinking Unsupervised Graph Anomaly Detection With Deep Learning: Residuals and Objectives. *IEEE Trans. Knowl. Data Eng.* 37, 2 (2025), 881–895.
- [14] Giang Hoang Nguyen, John Boaz Lee, Ryan A. Rossi, Nesreen K. Ahmed, Eunye Koh, and Sungchul Kim. 2018. Continuous-Time Dynamic Network Embeddings. In *Companion of the The Web Conference 2018 on The Web Conference 2018, WWW 2018, Lyon, France, April 23–27, 2018*. ACM, 969–976.
- [15] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao B. Schardl, and Charles E. Leiserson. 2020. EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence*. AAAI Press, 5363–5370.
- [16] Michael Poli, Stefano Massaroli, Junyoung Park, Atsushi Yamashita, Hajime Asama, and Jinkyoo Park. 2019. Graph Neural Ordinary Differential Equations. *CoRR abs/1911.07532* (2019).
- [17] Kieran Rendall, Alexios Mylonas, Stilianos Vidalis, and Dimitris Gritzalis. 2025. MIDAS: Multi-layered attack detection architecture with decision optimisation. *Comput. Secur.* 148 (2025), 104154.
- [18] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael M. Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. *CoRR abs/2006.10637* (2020).
- [19] Aravind Sankar, Yanhong Wu, Liang Gou, Wei Zhang, and Hao Yang. 2020. DySAT: Deep Neural Representation Learning on Dynamic Graphs via Self-Attention Networks. In *WSDM '20: The Thirteenth ACM International Conference on Web Search and Data Mining*. ACM, 519–527.
- [20] Sheng Tian, Jihai Dong, Jintang Li, Wenlong Zhao, Xiaolong Xu, Baokun Wang, Bowen Song, Changhua Meng, Tianyi Zhang, and Liang Chen. 2023. SAD: Semi-Supervised Anomaly Detection on Dynamic Graphs. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. ijcai.org, 2306–2314.
- [21] Rakshit Trivedi, Mehrdad Farajtabar, Prasenjeet Biswal, and Hongyuan Zha. 2019. DyRep: Learning Representations over Dynamic Graphs. In *7th International Conference on Learning Representations*. OpenReview.net.
- [22] Xuhong Wang, Ding Lyu, Mengjian Li, Yang Xia, Qi Yang, Xinwen Wang, Xinguang Wang, Ping Cui, Yupu Yang, Bowen Sun, and Zhenyu Guo. 2021. APAN: Asynchronous Propagation Attention Network for Real-time Temporal Graph Embedding. In *SIGMOD '21: International Conference on Management of Data*. ACM, 2628–2638.
- [23] Yanbang Wang, Yen-Yu Chang, Yunyu Liu, Jure Leskovec, and Pan Li. 2021. Inductive Representation Learning in Temporal Networks via Causal Anonymous Walks. In *9th International Conference on Learning Representations*. OpenReview.net.
- [24] Yuxia Wu, Yuan Fang, and Lizi Liao. 2024. On the Feasibility of Simple Transformer for Dynamic Graph Modeling. In *Proceedings of the ACM on Web Conference 2024*. ACM, 870–880.
- [25] Meghanath Macha Yadagiri, Deepak Pai, and Leman Akoglu. 2018. ConOut: Contextual Outlier Detection with Multiple Contexts: Application to Ad Fraud. In *Machine Learning and Knowledge Discovery in Databases - European Conference (Lecture Notes in Computer Science, Vol. 11051)*. Springer, 139–156.
- [26] Xiao Yang, Xuejiao Zhao, and Zhiqi Shen. 2025. A Generalizable Anomaly Detection Method in Dynamic Graphs. In *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence*. AAAI Press, 22001–22009.
- [27] Jiaxuan You, Tianyu Du, and Jure Leskovec. 2022. ROLAND: Graph Learning Framework for Dynamic Graphs. In *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 2358–2366.
- [28] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. 2023. Towards Better Dynamic Graph Learning: New Architecture and Unified Library. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023*.
- [29] Ziyuan Zhao, Peisheng Qian, Xulei Yang, Zeng Zeng, Cuntai Guan, Tam Wai Leong, and Xiaoli Li. 2023. SemiGNN-PPI: Self-Ensembling Multi-Graph Neural Network for Efficient and Generalizable Protein-Protein Interaction Prediction. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. ijcai.org, 4984–4992.

A Appendix

A.1 Detailed Dataset Description

Classic dynamic graph datasets

- (1) **Wikipedia**: A dynamic graph of Wikipedia page edits, where nodes represent pages and edges represent consecutive editing behaviors by users, capturing the evolutionary relationships of page collaborations.
- (2) **Reddit**: A dynamic graph of user posting interactions, where nodes represent users and edges represent consecutive posting or commenting interactions between users.

Blockchain financial networks

- (1) **Bitcoin-alpha**: A network of trust relationships among Bitcoin users, where nodes represent users and edges represent transaction trust ratings.
- (2) **Bitcoin-OTC**: An over-the-counter Bitcoin trading network, including dynamic changes in transaction frequency and amount between users.

Public time series datasets (UCR Archive)

- (1) **Earthquakes (EQS)**: Seismic wave monitoring time series data, detecting abnormal vibration patterns.
- (2) **WormsTwoClass (WTC)**: Time series of nematode movement trajectories, distinguishing between normal and abnormal movement behaviors.
- (3) **Strawberry (STB)**: Agricultural sensor time series data, identifying abnormal stages of crop growth.

Table 2: Classic Dynamic Graph Datasets and Blockchain Financial Networks

Dataset	Nodes	Anomaly Type	Pos. Ratio
Wikipedia	2,400	Malicious edits, vandalism	2.8%
Reddit	10,984	Spam posts, abnormal interactions	4.1%
Bitcoin-alpha	3,783	Fraudulent transactions	2.1%
Bitcoin-OTC	5,881	Abnormal trust relationships	1.9%

Table 3: Public Time Series Datasets

Dataset	Seq. Length	Anomaly Type	Pos. Ratio
EQS	512	Abnormal vibrations	25.3%
WTC	900	Biological behavior analysis	42.2%
STB	235	Crop growth	64.3%

A.2 Notations

Table 4: Time Series Transformer Notations

Notation	Description
v	Candidate shapelet, discriminative time series subsequence
w	Local temporal weights for alignment path in DTW operations
u	Global temporal weights for capturing periodic or phase changes
W_K	Time window for time series segmentation
$C(s)$	Shapelet set associated with segment s
γ	Time decay factor
$w_{p,q}^{(k)}$	Transition weight between adjacent segments

Table 5: Dynamic Memory Enhancer Notations

Notation	Description
$h^{(l)}(v, t_v)$	Temporal feature representation of node v in layer l
$\phi(t_v - t_u)$	Temporal difference feature between center node v and target node u
$Q(t), K(t), V(t)$	Query, Key and Value in attention mechanism
α_u	Contribution weight of neighbor node u to center node v
m_u, m_v	Message vectors for nodes u and v

Table 6: Anomaly Detector Notations

Notation	Description
$S_{lc}(v_i, t)$	Time contrastive score measuring degree of mutation in long-term patterns
$S_{mg}(v_i, t)$	Memory generation score evaluating deviation between short-term and long-term patterns
$S(v_i, t)$	Final anomaly score as weighted harmonic mean of both scores

A.3 Detailed Baseline Settings

Implementation Details. We implemented DyConAD using PyTorch and accelerated model training with an NVIDIA® RTX 3060

(24 GB) GPU, with PyCharm as the development environment. The loss function was optimized using the Adam optimizer.

In this section, we compare our proposed method with four categories of anomaly detection methods:

- (1) **Traditional anomaly detection methods:** SedanSpot [6], MIDAS [17], F-FADE [2]
- (2) **Incremental detection methods:** Anoedge
- (3) **Dynamic graph embedding methods:** JODIE [10], DyRep [21], TGAT [4], TGN [18]
- (4) **Dynamic memory storage methods:** SAD [20], SLADE [11]

SedanSpot (2018): A randomized algorithm for real-time detection of anomalies in edge streams, focusing on temporally evolving directed graphs. Its core lies in combining temporal and structural information to identify anomalous edges.

MIDAS (2020): Real-time statistical analysis of edge frequencies and node degree distributions to detect bursty anomalous edges. It processes edge stream data with low latency but is less sensitive to long-term pattern drift.

F-FADE (2021): Effectively models the temporal evolution distribution of interactions between node pairs through frequency factorization techniques, aiming to address the limitations of current methods in handling dynamic changes and lack of labels.

Anoedge (2023): Proposes four novel online methods for real-time detection of anomalous edges and graphs, featuring constant memory and update time.

JODIE (2019): Jointly learns temporal embeddings of users and items, detecting anomalies through embedding reconstruction errors, excelling in user-item interaction scenarios (e.g., recommendation systems).

DyRep (2019): Considers the dynamic changes in network topology and node interactions, decomposing dynamic networks into association processes and communication processes.

TGAT (2020): Proposes that node embeddings should include both static node features and changing topological features, developing temporal encoding techniques based on the classical Bochner theorem from harmonic analysis.

TGN (2020): Combines memory modules and graph convolution operations to propose a universal and efficient dynamic graph model capable of learning from the temporal sequence of data.

SAD (2023): Proposes an end-to-end semi-supervised anomaly detection framework that effectively addresses anomaly detection problems on dynamic graphs by introducing a temporally constrained repository and a pseudo-label contrastive learning module.

SLADE (2024): Proposes a self-supervised learning framework specifically designed for edge stream anomaly detection, effectively detecting dynamic anomalies without requiring labeled data.

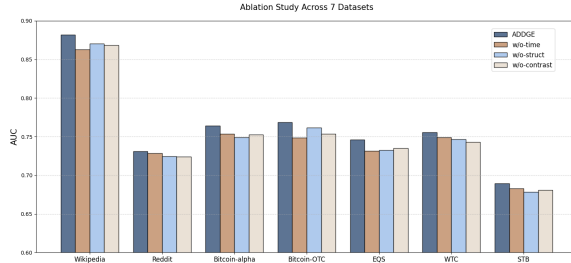
A.4 Ablation Study

How do different modules contribute to model performance? To assess the contribution of each module to the final performance of our model, we conducted ablation experiments by removing specific components. In this section, we designed three variants to evaluate how each module enhances DyConAD's performance, with notations shown in Table 7 and results presented in Figure 7.

The figure 7 demonstrates the ablation study results of the DyConAD model and its three variants (without temporal component,

Table 7: Description of Ablation Study Variants

Variant	Description
Ours-w/o-time	Node embeddings generated without temporal memory
Ours-w/o-struct	Node embeddings generated without structural memory
Ours-w/o-Contrast	Node embeddings generated without contrastive learning
DyConAD	The complete DyConAD model proposed in this paper

**Figure 7: Ablation Study.**

without structural component, and without contrastive learning component) across seven datasets. Overall, the complete DyConAD model achieved the best performance on all datasets, with AUC values consistently higher than all variant models. Particularly on the Wikipedia dataset, DyConAD reached an AUC value of approximately 0.88, significantly outperforming other variants; while on other datasets such as Bitcoin-alpha and Bitcoin-OTC, DyConAD maintained a lead of around 1 percentage point, proving that the

synergistic effect of all components is crucial for enhancing anomaly detection performance.

Further analysis of the contribution of different components reveals that the absence of the temporal component had the most significant impact on model performance, especially on Wikipedia and blockchain-related datasets, highlighting the importance of memory enhancement strategies. The absence of the structural component caused larger performance drops on the Bitcoin series datasets, while the absence of the contrastive learning component led to performance degradation across most datasets. These results fully demonstrate the rationality of the DyConAD model design, indicating that in dynamic graph anomaly detection, temporal sequence modeling, graph structural feature extraction, and contrastive learning mechanisms are all indispensable, collectively forming a complete framework for effectively capturing spatiotemporal anomaly patterns in complex networks.

A.5 Limitations

DyConAD primarily focuses on detecting node behavior anomalies and interaction pattern anomalies, but may not effectively identify certain specific types of graph anomalies, such as community structure changes or global graph property mutations. Although the paper proposes a method for converting time series to dynamic graphs, this conversion may not completely preserve all pattern features in the original time series data. Particularly when processing time series data with complex periodicity or nonlinear characteristics, the conversion process may lead to information loss.